LAPORAN PRAKTIKUM STRUKTUR DATA

“Praktikum Pekan 3”

OLEH:

LEXI MULIA YUNASPI

2511531006

MATA KULIAH STRUKTUR DATA
DOSEN PENGAMPU : DR WAHYUDI ,MT

ASISTEN PRAKTIKUM :M. GHALID

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum mata kuliah Struktur Data dengan judul “Praktikum Pekan 3” ini dapat terselesaikan dengan baik. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas praktikum pada Departemen Informatika, Fakultas Teknologi Informasi, Universitas Andalas.Dalam laporan ini, dibahas mengenai implementasi struktur data *Stack* menggunakan bahasa pemrograman Java di lingkungan Eclipse IDE. Fokus utama praktikum meliputi pembuatan class ADT untuk menyimpan data website serta simulasi riwayat penelusuran (*history*) pada *browser* melalui operasi *push*, *pop*, dan *peek*.

Penulis menyampaikan ucapan terima kasih kepada Bapak Dr. Wahyudi, MT selaku dosen pengampu mata kuliah Struktur Data atas bimbingan dan arahan yang telah diberikan selama proses perkuliahan dan praktikum. Penulis juga menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, kritik dan saran yang membangun sangat penulis harapkan demi perbaikan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca, khususnya mengenai penerapan struktur data dalam pengembangan perangkat lunak.

Padang, 19 April 2026

Penulis

DAFTAR ISI

KATA PENGANTAR…….……i

DAFTAR ISI…….….….....ii

BAB I PENDAHULUAN…….…..1

1.1 Latar Belakang…….....1

1.2 Manfaat…….….....1

1.3 Tujuan…….…...….....1

BAB II PEMBAHASAN…….,….……..2

2.1 Langkah Langkah Praktikum…….….…...2

2.2 Langkah Langkah Tugas Praktikum…….…19

BAB III PENUTUP……..…….20

3.1 Kesimpulan…….….….20

3.2 Saran…….…20

DAFTAR PUSTAKA…….....21

BAB I

PENDAHULUAN

1.1 Latar Belakang

truktur data merupakan cara penyimpanan, pengorganisasian, dan pengaturan data di dalam media penyimpanan komputer sehingga data tersebut dapat digunakan secara efisien. Salah satu struktur data linear yang penting adalah *Stack* (tumpukan). *Stack* bekerja dengan prinsip LIFO (*Last In First Out*), di mana data yang terakhir masuk akan menjadi yang pertama keluar. Implementasi *Stack* sangat luas dalam dunia pemrograman, salah satunya adalah untuk mengelola riwayat penelusuran (*history*) pada aplikasi *web browser*. Melalui praktikum ini, dipelajari bagaimana mengaplikasikan konsep *Stack* menggunakan bahasa pemrograman Java untuk mensimulasikan fungsi navigasi halaman *web*.

1.2 Manfaat

Praktikum ini memberikan manfaat bagi mahasiswa dalam memahami mekanisme kerja struktur data *Stack* secara nyata melalui pengkodean. Selain itu, mahasiswa dapat melatih logika pemrograman dalam menggunakan library standar Java (java.util.Stack) serta memahami penggunaan metode-metode utama seperti push, pop, dan peek dalam sebuah aplikasi terstruktur.

1.3 Tujuan

1. Memahami konsep dasar dan karakteristik struktur data *Stack* (LIFO).
2. Mampu mengimplementasikan operasi *push*, *pop*, dan *peek* menggunakan Java di lingkungan Eclipse IDE.
3. Mampu membangun program simulasi riwayat *browser* sederhana yang mengintegrasikan class ADT (Website) dengan class manajemen riwayat.

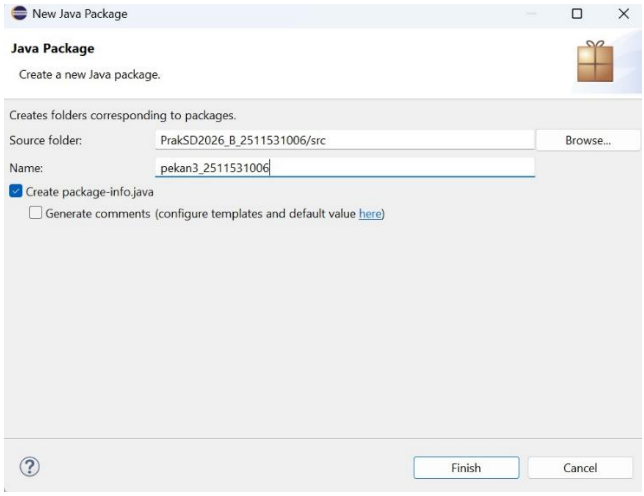
BAB II

PEMBAHASAN

2.1 Langkah Langkah Praktikum

1. Program 1

   A. Pada gambar tersebut ditampilkan proses pembuatan package baru di Eclipse IDE dengan nama *pekan3_2511531006*. Package ini berfungsi sebagai wadah untuk mengelompokkan class agar lebih terstruktur dan rapi dalam satu proyek. Opsi *create package-info.java* dicentang, yang berarti sistem akan otomatis membuat file informasi package, meskipun tidak wajib digunakan dalam program utama.



   B. Pada gambar pertama ditampilkan bagian awal kode program Java, yaitu deklarasi *package pekan3_2511531006* serta *import java.util.Stack* yang digunakan untuk memanfaatkan struktur data stack. Selain itu, didefinisikan class *LatihanStack_2511531006* beserta method *main* sebagai titik awal eksekusi program.

```java
package pekan3_2511531006;

import java.util.Stack;

public class latihanStack_2511531006 {

    public static void  main(String[] args) {
```

C. Pada gambar kedua ditunjukkan proses pembuatan objek stack dengan tipe data *Integer* menggunakan *Stack<Integer>*. Selanjutnya dilakukan operasi *push* untuk menambahkan beberapa data ke dalam stack, yaitu 42, -3, dan 17.

```java
    Stack<Integer> s = new Stack<Integer>();
    s.push(42);
    s.push(-3);
    s.push(17);
```

D. Pada gambar ketiga ditampilkan penggunaan operasi pada stack. Program mencetak isi stack, kemudian menggunakan *pop* untuk mengambil dan menghapus elemen teratas, serta *peek* untuk melihat elemen teratas tanpa menghapusnya. Hasil setiap operasi juga ditampilkan untuk menunjukkan perubahan isi stack.

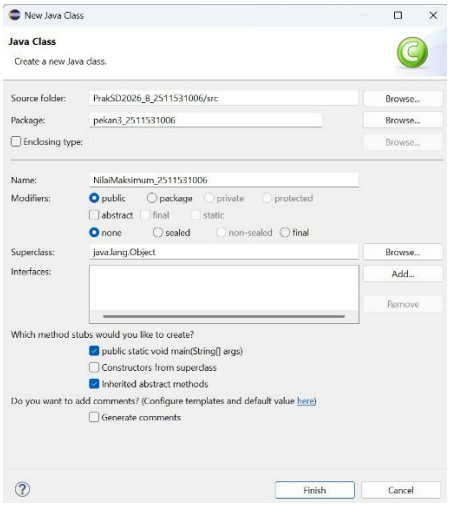
```java
    System.out.println("Nilai stack="+s);
    System.out.println("nilai pop="+s.pop());
    System.out.println("nilai stack setelah pop="+s);
    System.out.println("nilai peek"+s.peek());
    System.out.println("nilai stack setelah peek="+s);
    }

}
```

E. Output

```
Console
<terminated> latihanStack_2511531006 [Java Applicatic
Nilai stack=[42, -3, 17]
nilai pop=17
nilai stack setelah pop=[42, -3]
nilai peek-3
nilai stack setelah peek=[42, -3]
```

2. Program 2

A. Program dibuat di Eclipse IDE dengan membuat class *NilaiMaksimum_2511531006* di dalam package *pekan3_2511531006*. Class ini dilengkapi method *main* sehingga dapat dijalankan sebagai program utama.



B. Pada bagian awal kode, dilakukan deklarasi *package* dan *import java.util.Stack* untuk menggunakan struktur data stack. Kemudian didefinisikan class beserta method tambahan max(Stack<Integer> s) yang berfungsi untuk mencari nilai maksimum dalam stack.

```
1 package pekan3_2511531006;
2
3 import java.util.Stack;
4
5 public class NilaiMaksimum_2511531006 {
6
```

C. Di dalam method max, elemen pertama stack diambil sebagai nilai awal maksimum, lalu semua elemen diproses satu per satu menggunakan perulangan. Setiap nilai dibandingkan dengan Math.max untuk mendapatkan nilai terbesar, sambil dipindahkan ke stack sementara (*backup*). Setelah proses selesai, seluruh isi stack dikembalikan lagi ke kondisi semula.

```java
    public static int max (Stack<Integer> s) {
        Stack<Integer> backup = new Stack<Integer> ();
        int maxValue_2511531006 = s.pop();
        backup.push(maxValue_2511531006);
        while (!s.empty()) {
            int next = s.pop();
            backup.push(next);
            maxValue_2511531006 = Math.max(maxValue_2511531006, next);
        }
```

D. Pada bagian *main*, dibuat objek *Stack<Integer>* dan diisi beberapa data menggunakan operasi *push*. Program kemudian menampilkan isi stack untuk memastikan data yang dimasukkan.

```java
        while (!backup.isEmpty()) {
            s.push(backup.pop());
        }
        return maxValue_2511531006;
    }
```

E. Terakhir, program memanggil method max(s) untuk mendapatkan nilai maksimum dari stack.

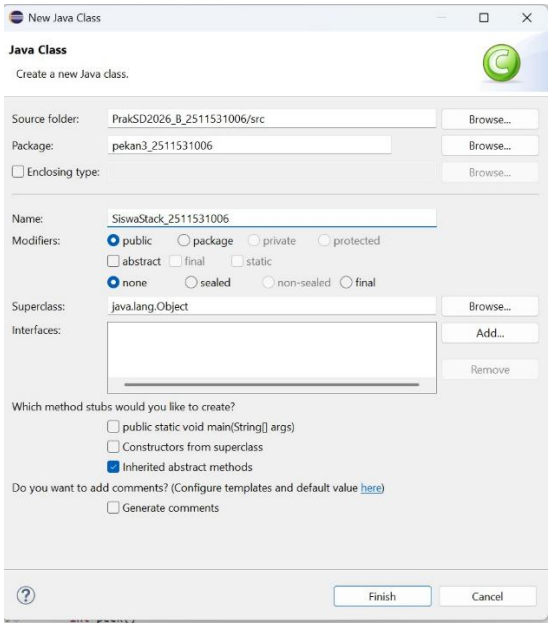
```java
    public static void main (String[] args) {
        Stack<Integer> s = new Stack<Integer>();
        s.push(70);
        s.push(12);
        s.push(20);
        System.out.println("isi stack");
        System.out.println("Stack Teratas"+s);
        System.out.println("Nilai Maksimum"+max(s));
        }


    }
```

F. Output

```
Console
<terminated> NilaiMaksimum_2511531006 [Java Application]
isi stack
Stack Teratas[70, 12, 20]
Nilai Maksimum70
```

3. Program 3

A. Program di Eclipse IDE diawali dengan pembuatan class *SiswaStack_2511531006* dalam package *pekan3_2511531006*. Class ini digunakan untuk merepresentasikan data mahasiswa dalam program.



B. Dilakukan import *ArrayList* sebagai struktur penyimpanan data. Hal ini digunakan untuk menyimpan kumpulan objek secara dinamis.

```
1  package pekan3_2511531006;
2
3  import java.util.ArrayList;
4
5  public class SiswaStack_2511531006 {
```

C. Class memiliki atribut *nama* dan *nim* sebagai data utama. Atribut ini menyimpan identitas dari setiap mahasiswa.

```
6      String nama;
7      int nim;
8
9      public SiswaStack_2511531006(String nama, int nim) {
10         this.nama = nama;
11         this.nim = nim;
12     }
13
```

D. Constructor digunakan untuk menginisialisasi nilai atribut. Sehingga setiap objek langsung memiliki data saat dibuat.

```
    @Override
    public String toString() {
        return "Nim: "+ nim +", Nama: " +nama;
    }
}
```

E. Method *toString()* digunakan untuk menampilkan data. Output ditampilkan dalam bentuk teks yang mudah dibaca.

```
class SiswaStack {
    private ArrayList<SiswaStack_2511531006> stack;

    public SiswaStack() {
        stack = new ArrayList<>();
    }

    public void push(SiswaStack_2511531006 mhs) {
        stack.add(mhs);
    }

    public SiswaStack_2511531006 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() -1);
        }
        return null;
    }
```

F. Selanjutnya dibuat class *SiswaStack* sebagai struktur stack. Class ini mengatur proses penyimpanan data mahasiswa.

```
    public SiswaStack_2511531006 peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }
```

G. Atribut *stack* menggunakan *ArrayList* sebagai media penyimpanan. Data disusun mengikuti konsep stack (LIFO).

```java
    public static void main(String[] args) {
        SiswaStack studentStack = new SiswaStack();

        SiswaStack_2511531006 mhs1 = new SiswaStack_2511531006("Ali", 1);
        SiswaStack_2511531006 mhs2 = new SiswaStack_2511531006("Boby", 2);
        SiswaStack_2511531006 mhs3 = new SiswaStack_2511531006("Charles", 3);

```

H. Method *push*, *pop*, dan *peek* digunakan sebagai operasi utama. Fungsinya untuk menambah, mengambil, dan melihat data teratas.

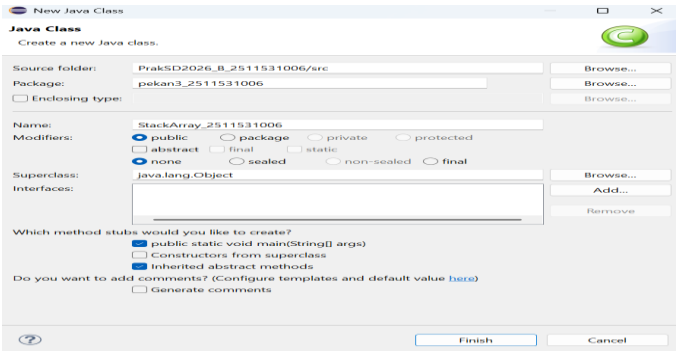
```java
        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

```

I. Method *isEmpty* dan *tampilkanSiswa* melengkapi fungsi stack. Digunakan untuk mengecek kondisi dan menampilkan seluruh isi stack.

```java
        System.out.println("Siswa di dalam stack:");
        System.out.println("mengeluarkan siswa teratas dari stack: " + studentStack.pop());
        System.out.println("daftar siswa setelah di stack:");
        studentStack.tampilkanSiswa();
    }
}
```

4. Program 4

A. Program di Eclipse IDE diawali dengan pembuatan class *stackArray_2511531006* dalam package *pekan3_2511531006*. Class ini digunakan untuk mengimplementasikan struktur data stack berbasis array.



B. Pada bagian awal kode, ditentukan kapasitas maksimum stack dengan variabel *MAX*.Selain itu terdapat variabel *top* dan array *a[]* sebagai tempat penyimpanan data.

```
package pekan3_2511531006;

public class stackArray_2511531006 {
```

C. Method *isEmpty()* digunakan untuk mengecek apakah stack kosong. Kondisi kosong terjadi ketika nilai *top* kurang dari 0.

```
        static final int MAX = 1000;
        int top;
        int a[] = new int [MAX];
```

D. Constructor digunakan untuk menginisialisasi nilai *top* menjadi -1. Hal ini menandakan bahwa stack masih kosong saat pertama dibuat.

```
        boolean isEmpty()
        {
            return (top < 0);
        }
        stackArray_2511531006()
        {
            top = -1;
        }
        boolean push (int x)
```

E. Method *push* digunakan untuk menambahkan elemen ke dalam stack. Jika stack penuh maka akan terjadi *overflow*, jika tidak data akan dimasukkan.

```
        {
            if (top >= (MAX - 1)) {
                System.out.println("Stack Overflow");
                return false;
            }
            else {
                a [++top] = x;
                System.out.println(x + " dimasukkan dalam stack");
                return true;
            }
        }
        int pop()
```

F. Method *pop* digunakan untuk mengambil dan menghapus elemen teratas. Jika stack kosong maka terjadi *underflow*, jika tidak elemen dikembalikan.

```
        {
            if (top < 0) {
                System.out.println("Stack Underflow");
                return 0;
            }
            else {
                int x = a[top--];
                return x;
            }

        }

    int peek()
```

G. Method *peek* digunakan untuk melihat elemen teratas tanpa menghapusnya. Jika kosong akan menampilkan pesan, jika tidak mengembalikan nilai teratas.

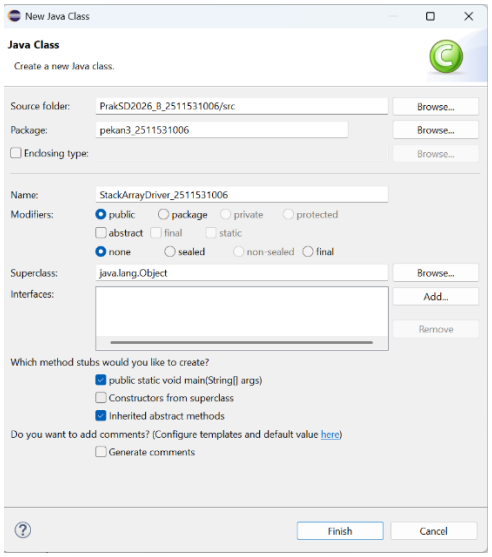
```
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
        else {
            int x = a [top];
            return x;
        }
    }
```

H. Method *print* digunakan untuk menampilkan seluruh isi stack. Data ditampilkan dari atas ke bawah sesuai konsep LIFO.

```
    void print() {
    for(int i = top;i>-1;i--) {
        System.out.print(""+ a[i]);
    }
}
}
```

5. Program 5

A. Program di Eclipse IDE membuat class *StackArrayDriver_2511531006* sebagai program utama.Class ini berada dalam package *pekan3_2511531006* dan memiliki method *main*.



B. Pada bagian kode awal ditampilkan deklarasi *package* dan class. Method *main* digunakan sebagai titik awal eksekusi program.

```
1  package pekan3_2511531006;
2
3  public class stackArrayDriver_2511531006 {
4
5      public static void main(String[] args) {
```

C. Selanjutnya dibuat objek *stackArray_2511531006* dan dilakukan operasi *push*. Data yang dimasukkan ke dalam stack adalah 10, 20, dan 30.

```
6          stackArray_2511531006 s= new stackArray_2511531006();
7          s.push(10);
8          s.push(20);
9          s.push(30);
```

D. Program kemudian melakukan *pop* dan *peek* serta menampilkan hasilnya. Output menunjukkan elemen yang keluar dan elemen teratas pada stack.

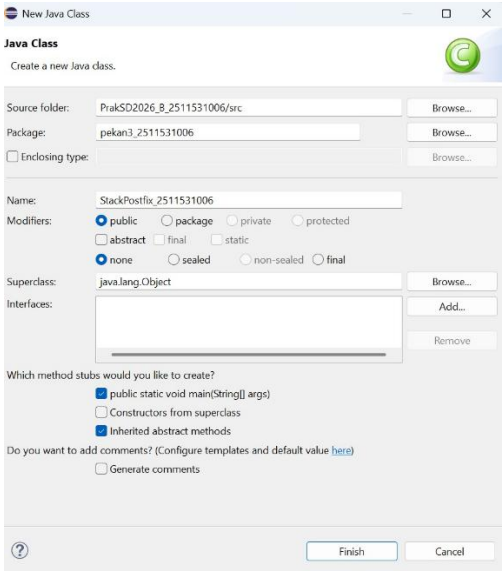
```
10         System.out.println(s.pop() +"dikeluarkan dari stack");
11         System.out.println("Elemen teratas adalah :" + s.peek());
12         System.out.println("Elemen pada stack :");
13     }
14
15 }
```

E. Output

```
Console
<terminated> stackArrayDriver_2511531006 [Java App
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30dikeluarkan dari stack
Elemen teratas adalah :20
Elemen pada stack :
```

6. Program 6

   A. Program di Eclipse IDE membuat class *StackPostfix_2511531006* sebagai program utama.Class ini berada dalam package *pekan3_2511531006* dan menggunakan *Scanner* untuk membaca ekspresi.



   B. Pada bagian kode awal terdapat deklarasi *package*, import *Scanner*, dan definisi class.Selain itu dibuat method postfixEvaluate untuk menghitung ekspresi postfix.

```
1  package pekan3_2511531006;
2
3⊕ import java.util.Scanner;
5
6  public class StackPostfix_2511531006 {
```

C. Di dalam method tersebut digunakan *Stack* untuk menyimpan operand. Scanner membaca input per token dari ekspresi yang diberikan.

```
    public static int postfixEvaluate(String expression) {
        Stack<Integer> s = new Stack<Integer>();
        Scanner input_2511531006 = new Scanner(expression);

```

D. Jika token berupa angka maka dimasukkan ke stack dengan *push*. Jika berupa operator, maka dua nilai diambil dari stack untuk dihitung.

```
        while (input_2511531006.hasNext()) {
            if (input_2511531006.hasNextInt()) {
                s.push(input_2511531006.nextInt());
```

E. Operasi yang digunakan meliputi penjumlahan, pengurangan, perkalian, dan pembagian.Hasil perhitungan dimasukkan kembali ke dalam stack.

```
            } else {
                String operator = input_2511531006.next();
                int operand2_2511531006 = s.pop();
                int operand1_2511531006 = s.pop();
                if (operator.equals("+")) {
                    s.push(operand1_2511531006 + operand2_2511531006);
                } else if (operator.equals("-")) {
                    s.push(operand1_2511531006 - operand2_2511531006);
                } else if (operator.equals("*")) {
                    s.push(operand1_2511531006 * operand2_2511531006);
                } else {
                    s.push(operand1_2511531006 / operand2_2511531006);
                }
            }
        }
```

F. Setelah semua proses selesai, stack akan berisi satu hasil akhir. Nilai tersebut diambil dan dikembalikan sebagai hasil evaluasi postfix.

```
        input_2511531006.close();
        return s.pop();
    }

```

G. Pada method *main*, program memanggil fungsi dengan contoh ekspresi postfix.

```java
    public static void main(String[] args) {
        System.out.println("Hasil postfix= " + postfixEvaluate("5 2 4 * 7 -"));
    }
}
```

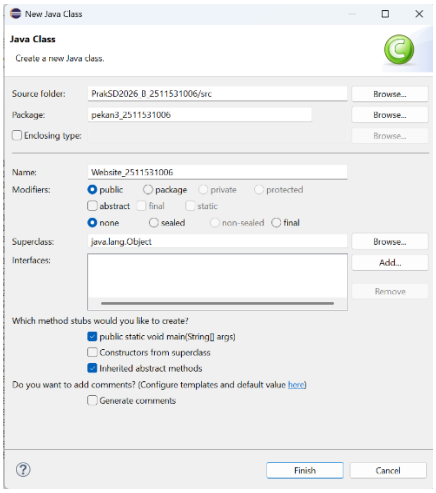
H. Output

```
Console
<terminated> StackPostfix_2511531006 [Java Application] C:\
Hasil postfix= 1
```

2.2 Langkah Langkah Tugas Praktikum

1. Program 1

A. Program di Eclipse IDE membuat class *Website_2511531006* dalam package *pekan3_2511531006*.Class ini digunakan sebagai ADT untuk menyimpan data website.



B. Pada bagian awal kode terdapat deklarasi *package* dan definisi class. Class menjadi dasar untuk merepresentasikan objek website.

```java
package pekan3_2511531006;

public class Website_2511531006 {
```

C. Atribut yang digunakan adalah *judul_1006* dan *url_1006*. Kedua atribut ini menyimpan informasi judul dan alamat website.

```java
    private String judul_1006;
    private String url_1006;
```

D. Constructor digunakan untuk menginisialisasi nilai atribut saat objek dibuat. Nilai parameter langsung disimpan ke dalam atribut class.

```java

    /**
     * Constructor untuk menginisialisasi atribut
     */
    public Website_2511531006(String judul_1006, String url_1006) {
        this.judul_1006 = judul_1006;
        this.url_1006 = url_1006;
    }

```

E. Method *getter* untuk judul digunakan untuk mengambil nilai *judul_1006*. Method ini mengembalikan data tanpa mengubah isi atribut.

```java
    /**
     * Getter untuk judul website
     */
    public String getJudul_1006() {
        return judul_1006;
    }

```

F. Method *getter* untuk URL digunakan untuk mengambil nilai *url_1006*. Digunakan untuk mengakses alamat website dari objek.

```java
    /**
     * Getter untuk URL website
     */
    public String getUrl_1006() {
        return url_1006;
    }

```

G. Method *setter* pada gambar pertama berfungsi untuk mengisi atau mengubah data pada atribut judul_1006 menggunakan parameter String yang diterima.

```java
    /**
     * Setter untuk judul website
     */
    public void setJudul_1006(String judul_1006) {
        this.judul_1006 = judul_1006;
    }

```

H. Method *setter* pada gambar kedua memiliki fungsi yang sama namun spesifik untuk memperbarui data alamat pada atribut url_1006 di dalam objek.

```java
    /**
     * Setter untuk URL website
     */
    public void setUrl_1006(String url_1006) {
        this.url_1006 = url_1006;
    }

```

I. Method toString pada gambar ketiga bertugas menggabungkan data judul dan URL tersebut menjadi satu format teks rapi agar informasi objek mudah ditampilkan.

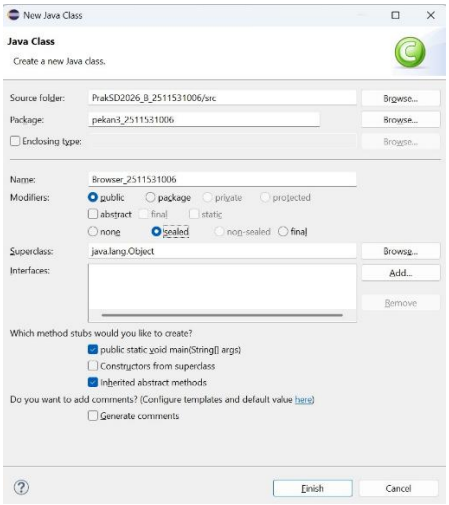
```java
    /**
     * Method toString untuk menampilkan informasi website
     */
    @Override
    public String toString() {
        return "Judul: " + judul_1006 + "\nURL: " + url_1006;
    }
}
```

J. Output

```
Console
Browser_2511531006 [Java Application] C:\Users\Hype AMD\.p
=== Browser History NIM: 2511531006 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. keluar
Pilihan: 1

Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login

Berhasil mengunjungi halaman!
```

2. Program 2

A. Program dibuat menggunakan Eclipse IDE dengan nama class Browser_2511531006 yang berada di dalam package pekan3_2511531006.



B. Bagian awal kode menyertakan deklarasi package dan import library java.util.Stack serta java.util.Scanner untuk kebutuhan struktur data dan input.

```
1  package pekan3_2511531006;
2
3  import java.util.Stack;
4  import java.util.Scanner;
5
6  public class Browser_2511531006 {
```

C. Atribut history_1006 dideklarasikan dengan tipe data Stack untuk menyimpan kumpulan objek dari class Website_2511531006.

```
7      private Stack<Website_2511531006> history_1006;
8
9      /**
10      * Constructor untuk menginisialisasi stack
11      */
```

D. Constructor class digunakan untuk menginisialisasi objek tumpukan (*stack*) baru agar siap digunakan saat program dijalankan.

```
12     public Browser_2511531006() {
13         history_1006 = new Stack<>();
14     }
```

E. Method kunjungiWebsite_1006 berfungsi untuk mensimulasikan proses membuka halaman web baru atau melakukan operasi *push* pada stack.

```java

    /**
     * Method untuk mengunjungi website (PUSH)
     */
```

F. Di dalam method tersebut, objek Scanner digunakan untuk mengambil input teks berupa judul dan URL secara interaktif dari pengguna.

```java
    public void kunjungiWebsite_1006() {
        Scanner input = new Scanner(System.in);

        System.out.print("Masukkan Judul: ");
        String judul = input.nextLine();

        System.out.print("Masukkan URL: ");
        String url = input.nextLine();

```

G. Data yang diinputkan kemudian dibentuk menjadi objek Website baru dan dimasukkan ke dalam tumpukan riwayat penelusuran.

```java
        // Membuat objek Website dan push ke stack
        Website_2511531006 website = new Website_2511531006(judul, url);
        history_1006.push(website);

        System.out.println("\nBerhasil mengunjungi halaman!");
        System.out.println("=== Halaman Aktif ===");
        System.out.println(website);
        System.out.println("==\n");
    }

```

H. Method tombolBack_1006 menerapkan operasi *pop* untuk menghapus dan mengambil data terakhir guna mensimulasikan navigasi kembali.

```java
    /**
     * Method Tombol Back (POP) - dengan pengecekan stack kosong
     */
    public void tombolBack_1006() {
        if (history_1006.isEmpty()) {
            System.out.println("History kosong! Tidak ada halaman untuk kembali.\n");
            return;
        }

```

I. Terdapat pengecekan kondisi isEmpty() untuk memastikan sistem tidak mengalami error saat mencoba mengakses riwayat yang masih kosong.

```java
        Website_2511531006 halaman = history_1006.pop();
        System.out.println("Kembali dari halaman:");
        System.out.println(halaman);
        System.out.println("Halaman berhasil ditutup!\n");
    }

```

J. Method lihatHalamanAktif_1006 menggunakan operasi *peek* untuk menampilkan data teratas tanpa menghapus halaman tersebut dari riwayat kunjungan.

```java
    /**
     * Method Lihat Halaman Aktif (PEEK) - dengan pengecekan stack kosong
     */
    public void lihatHalamanAktif_1006() {
        if (history_1006.isEmpty()) {
            System.out.println("History kosong! Tidak ada halaman aktif.\n");
            return;
        }

        Website_2511531006 halamanAktif = history_1006.peek();
        System.out.println("\n=== Halaman Aktif Saat Ini ===");
        System.out.println(halamanAktif);
        System.out.println("==\n");
    }

```

K. Output

```
Console
Browser_2511531006 [Java Application] C:\Users\Hype AMD\.p
=== Browser History NIM: 2511531006 ===

1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. keluar
Pilihan: 1

Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login

Berhasil mengunjungi halaman!
```

BAB III

PENUTUP

1.1 Kesimpulan

Berdasarkan hasil praktikum, dapat disimpulkan bahwa struktur data *Stack* sangat efektif digunakan untuk permasalahan yang membutuhkan urutan pemrosesan terbalik, seperti fitur "kembali" (*back*) pada *browser*. Operasi push berhasil digunakan untuk menambah riwayat kunjungan, pop untuk kembali ke halaman sebelumnya dengan menghapus data teratas, dan peek untuk melihat halaman aktif saat ini. Penggunaan kondisi isEmpty() sangat krusial untuk mencegah terjadinya *EmptyStackException* saat tumpukan kosong.

1.2 Saran

Untuk pengembangan selanjutnya, disarankan agar program simulasi ini ditambahkan fitur batas maksimal tumpukan (*stack limit*) untuk mensimulasikan penggunaan memori yang terbatas. Selain itu, integrasi dengan antarmuka grafis (GUI) akan membuat simulasi navigasi *browser* menjadi lebih interaktif dan mudah dipahami oleh pengguna**.**

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). *Data Structures and Algorithms in Java*. 6th Edition. Wiley.

Horowitz, E., Sahni, S., & Anderson-Freed, S. (2007). *Fundamentals of Data Structures in C*. 2nd Edition. Silicon Press.

Liang, Y. D. (2020). *Introduction to Java Programming and Data Structures*. 12th Edition. Pearson.